

# Documentação Técnica — Requisição Fácil

## Integração e Funcionamento

O sistema Requisição Fácil foi desenvolvido como uma aplicação web completa utilizando Django, aproveitando toda a infraestrutura robusta deste framework. Essa escolha estratégica proporcionou diversos benefícios significativos para o projeto, garantindo uma solução escalável e segura.

### Benefícios da Arquitetura

A escolha do Django como framework principal trouxe vantagens substanciais para o desenvolvimento:

#### Facilidade de Implementação

- **Framework maduro:** O Django possui uma arquitetura consolidada com ORM integrado, sistema de autenticação robusto e painel administrativo automático
- **Padrão MVT:** A estrutura Model-View-Template facilita a organização do código e sua manutenção contínua
- **Bibliotecas integradas:** Funcionalidades essenciais como validação de formulários, proteção CSRF e sistema de sessões já estão incluídas

#### Segurança Nativa

- **Proteção automática:** O Django oferece proteção contra ataques comuns (SQL Injection, XSS, CSRF) por padrão
- **Sistema de autenticação:** Controle de usuários e permissões já implementado e configurado
- **Validação de dados:** Sistema robusto de validação em múltiplas camadas

#### Escalabilidade e Performance

- **Cache integrado:** Sistema de cache flexível para otimização de performance
- **Conexão com Oracle:** Suporte nativo para bancos de dados corporativos
- **Deployment facilitado:** Compatibilidade com diversos servidores web

## Como Usar

### Acesso ao Sistema

Para execução local, acesse: <http://localhost:8000/requisicaoofacil/>

## Fluxo de Trabalho

1. **Dashboard:** Visualização das requisições atuais organizadas em ordem alfabética
2. **Validade:** Exibição dos itens com data de expiração próxima

## Desafios Técnicos

### Compatibilidade de Versões

O projeto utiliza Django 3.2 LTS, que apresenta os seguintes desafios técnicos:

- **Dependências específicas:** Algumas bibliotecas requerem versões exatas para manter compatibilidade
- **Migrações complexas:** Mudanças no ORM entre versões podem impactar migrações existentes

### Soluções Implementadas

# requirements.txt - Versões fixadas

cx\_Oracle==8.3.0

Django==3.2.25

django-cors-headers==4.5.0

### Estratégias de Versionamento:

- Ambiente virtual isolado para evitar conflitos com outras aplicações
- Documentação completa de dependências para rastreamento de bibliotecas utilizadas

### Integração Python/Apache

A configuração do servidor apresenta complexidades específicas na integração Python/Apache através do mod\_wsgi.

### Desafios de Deployment

- **Configuração do mod\_wsgi:** Necessita compilação específica para a versão do Python utilizada
- **Permissões de arquivo:** Configuração adequada de usuários e grupos do sistema
- **Variáveis de ambiente:** Propagação correta para o ambiente Apache
- **Logs distribuídos:** Gerenciamento eficiente de logs entre Django e Apache

# Guia de Manutenção

## Preparação do Ambiente

Para colaborar no projeto, sua estação de trabalho deve estar configurada com os seguintes componentes:

### Requisitos do Sistema

- **Python 3.10+:** Ambiente de execução principal
- **Apache 2.4+:** Servidor web para ambiente de produção
- **Git:** Sistema de controle de versões
- **Visual Studio Code:** Editor recomendado (opcional)

### Configuração no Linux

# Atualizar sistema

```
sudo apt-get update && sudo apt-get upgrade
```

# Instalar dependências

```
sudo apt-get install python3 python3-pip python3-venv
```

```
sudo apt-get install apache2 libapache2-mod-wsgi-py3
```

```
sudo apt-get install git
```

# Oracle Instant Client

# Instalação direta

```
wget
```

```
https://download.oracle.com/otn_software/linux/instantclient/2110000/instantcli  
ent-basic-linux.x64-21.10.0.0.0dbru.zip
```

```
sudo unzip instantclient-basic-linux.x64-21.10.0.0.0dbru.zip -d /opt/oracle/
```

# Alternativamente, clonar o repositório do projeto

```
gh repo clone ViniciusBenevides/Projetos-JF --branch api_oracle-env
```

### Configuração do Projeto

# Clonar repositório

```
gh repo clone ViniciusBenevides/Projetos-JF --branch requisicaofacil-env
```

# Criar ambiente virtual

```
python3 -m venv requisicaofacil-env
```

```
cd requisicaofacil-env
```

```
source bin/activate
```

# Instalar dependências

```
pip install -r requirements.txt
```

```
# Acessar o projeto Django  
cd requisicaofacil
```

## Configurações do Sistema

### Endereçamento do Cliente Oracle

Após instalar o projeto Django, é necessário configurar o endereçamento do cliente Oracle.

#### Endereçamento Local

Para permitir que o `runserver` local utilize o cliente Oracle, altere em `settings.py`:

```
oracle_home =  
    "/var/www/html/sistemas/api_oracle-env/api_oracle/oracle/instantclient_23_7"
```

Substitua pelo endereço do cliente Oracle da máquina atual.

#### Endereçamento no Apache

Para configurar o cliente Oracle no Apache2:

1. Acesse o arquivo `envvars`:

```
cd /etc/apache2
```

2. Adicione as seguintes linhas ao arquivo:

```
# Endereços para o Oracle  
export ORACLE_HOME="/<caminho para a pasta oracle>/instantclient_23_7"  
export LD_LIBRARY_PATH="/<caminho para a pasta oracle>/instantclient_23_7"  
export PATH="/<caminho para a pasta oracle>/instantclient_23_7:$PATH"
```

**Importante:** Substitua `<caminho para a pasta oracle>` pelo caminho real da instância Oracle instalada.

**Nota sobre Permissões:** Garanta que o programa tenha permissão de acesso à pasta Oracle. Para execução local, são utilizadas as permissões do usuário atual;

para execução via Apache, são utilizadas as permissões do usuário `www-data` por padrão.

## Estrutura e Funcionalidades

### Estrutura do Projeto Django

```
requisicaofacil/
├── app1/           # Landing page (em desuso)
├── consultaOficial/ # Aplicativo responsável pelas queries
├── requisicaofacil/ # Configurações do Django (settings, wsgi, etc.)
└── db.sqlite3      # Banco de dados para armazenar apenas os models
```

### Módulos Principais

#### 1. app1

- `views.py`: Views genéricas reutilizáveis

#### 2. consultaOficial

- `models.py`: Modelo de requisição e relacionados
- `views.py`: Views específicas para as requisições

### Configurações de Banco de Dados

```
# settings.py - Configurações de Banco
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    },
    'oracle': {
        'ENGINE': 'django.db.backends.oracle',
        'NAME': 'JFGO',
        'USER': 'go2448ps',           # Sujeito a mudanças
        'PASSWORD': 'Tjpyfm98.',     # Sujeito a mudanças
        'HOST': '172.22.3.3',
        'PORT': '1521',
    },
}
```